

Contents

Project Overview

Explored Strategies

Challenges

Pipeline

Outcome

Small Demo?

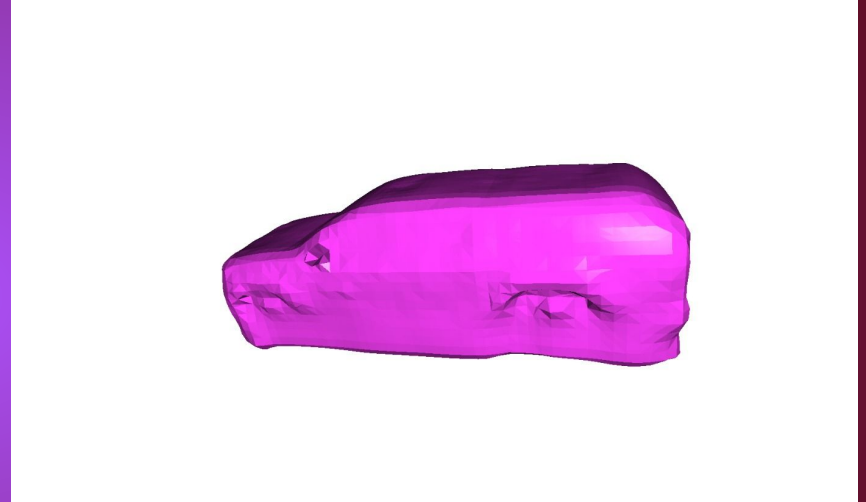
The Lab 2026

Brushless Car Wash - 3D Reconstruction

The Three

Rufina Chyhohidze, Lucas Belmans, Filipe Kososki Guimaraes

Make car wash brushless



Project Goal

Transform raw .pcd point cloud data into an accurate 3D mesh (STL).

Requirements:

Preserve important vehicle details such as:

- Mirrors
- Wheels
- Tow Bar
- Bull Bar
- Roof

Final Objective:

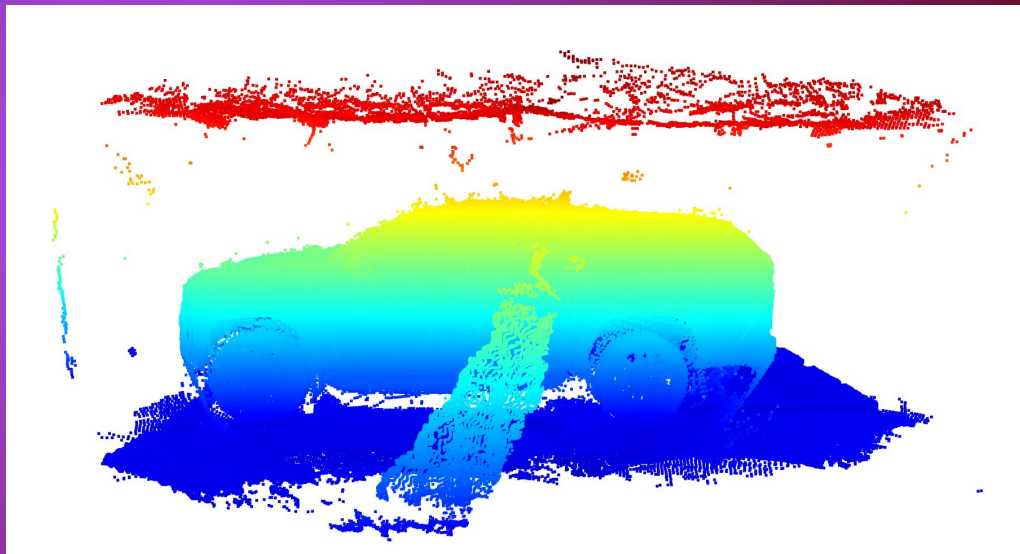
Accurate 3D mesh that can be safely used in brushless car washing.

Problem statement

Raw point cloud data contains large amounts of unwanted data.

What do we mean
by unwanted data?

- Floor
- Ceiling
- Isolated clusters
- Walls
- Statistical outliers



Challenge

Remove noise while preserving important car features.

Work plan

1. Extract Raw Point Cloud data
2. Remove all detected noise
3. Detect important car features
4. Handle abnormal or unsafe data conditions
5. Convert the processed point cloud into a watertight 3D mesh
6. Export .stl

Tech stack:

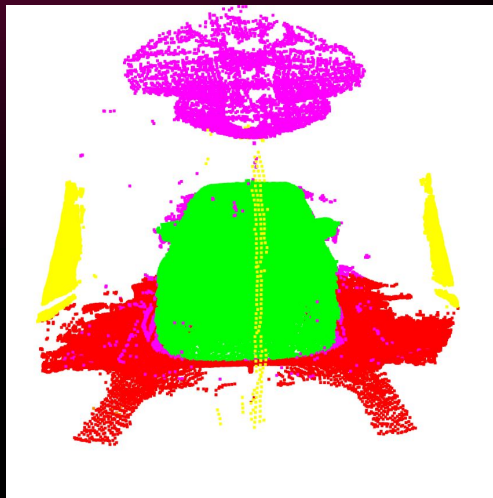
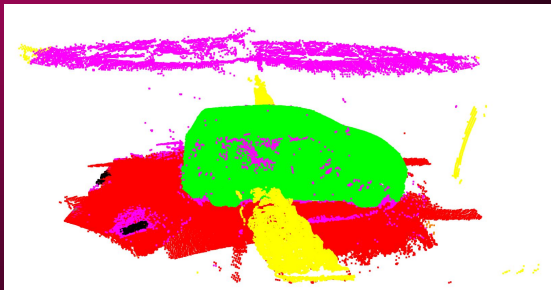
Python

Libraries:

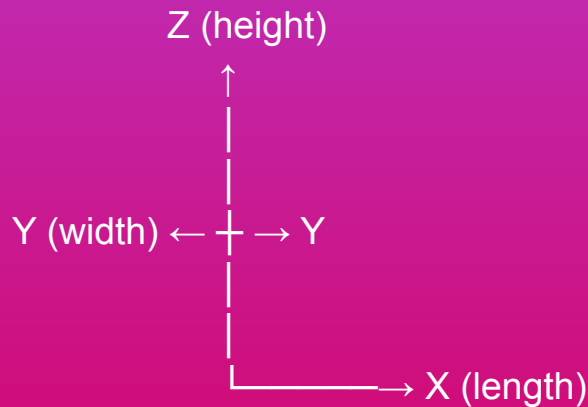
- Open 3d
- TryMesh
- ShapeLi
- PyMesh
- PyMeshfix
- Meshlib

Visualisation tool:
Blender, Open 3d
visualisation tool

Filtering mechanism



To isolate the car from the raw point cloud, we implemented a multi-stage filtering pipeline combining geometric detection and statistical analysis



Filtering mechanism overview

Step 1 - Downsampling

Reduce data density - Voxel

- Improve processing speed
- Stabilize plane detection
- Reduce noise sensitivity

Step 3 - Geometric filtering

- Side walls (bounding box margins)
- Below curtain points (height threshold)

Step 2 - Plane Detection

Detect large horizontal planes - RANSAC

- Floor
- Ceiling

Step 4 - Statistical Outliers + Cluster filtering

Distance to neighbors $>$ mean + std deviation threshold

Parameters: Neighbour count + std ratio

Explored Strategies

Meshing

Alpha Shape reconstruction

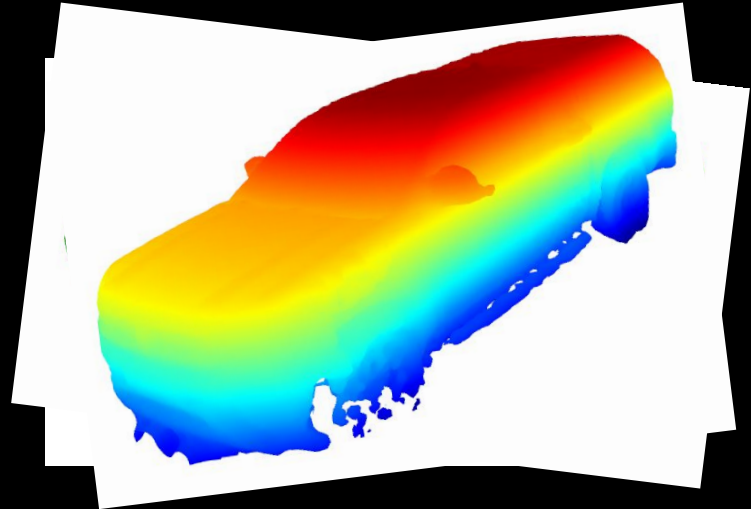
“Cover” the car

Poisson reconstruction

Voxel + normal (vector) math

Ball Pivoting reconstruction

Roll a “ball” over the car



Smoothing

Average Filter smoothing

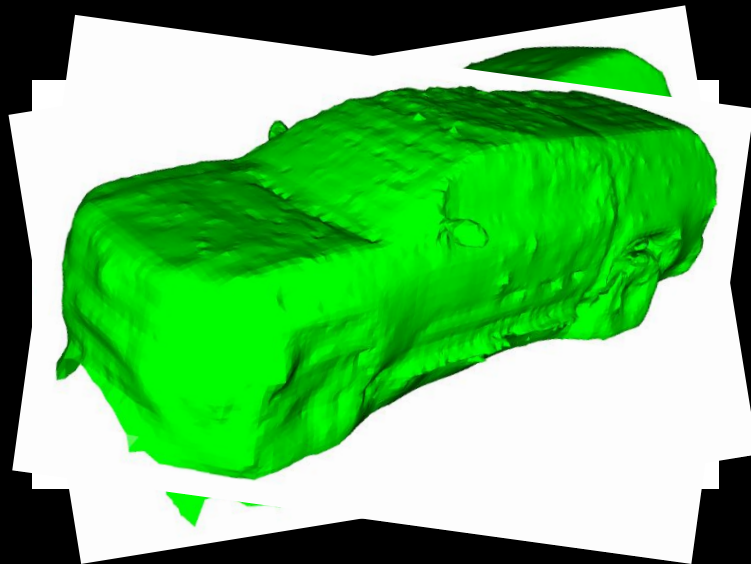
Average vertices

Downsampling smoothing

Downsample before meshing
"Averaging points"

Laplacian Filter smoothing

Vertices with weights



Keeping Details

Detecting Sides with Planes

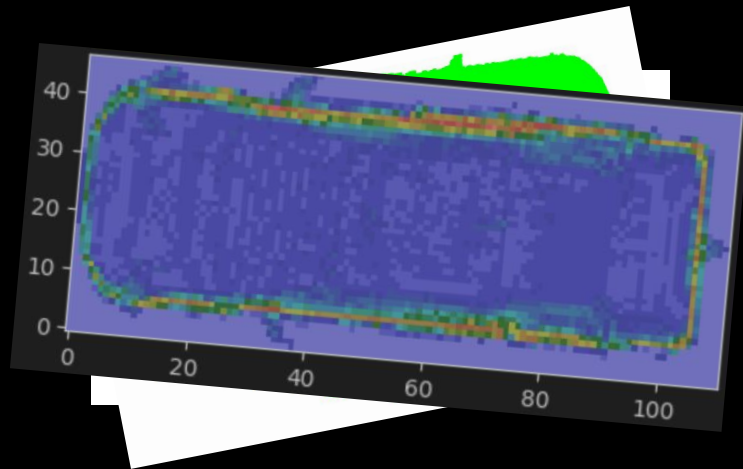
Extract estimated sides of car by fitting planes to them and cutting at the plane

Detecting Sides with Coordinates

Extract estimated sides of car by distance from side of the pointcloud and cutting at that distance

Detecting Sides with Density

Extract estimated sides of car by density on z plane of the pointcloud and cutting at those x/y coordinates



Smooth "Curtain"

Curtain with Cutting

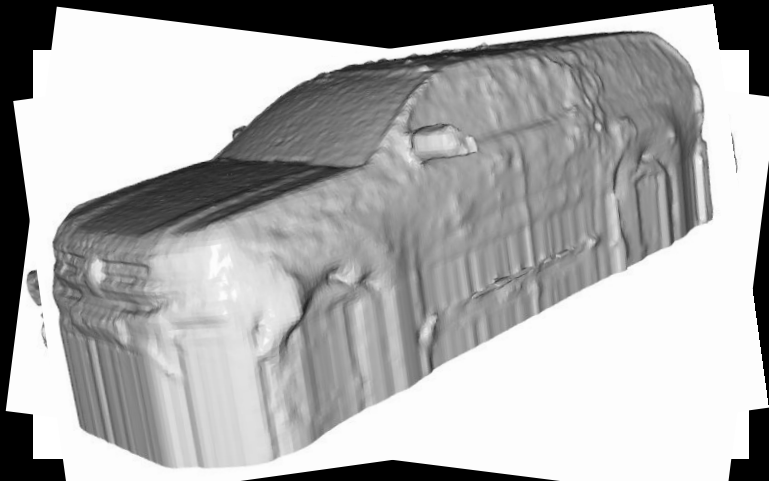
Cut mesh at calculated height
Pull curtain down from cut car

Curtain without Cutting

Pull curtain down from calculated height

Curtain with Interior Removal

Pull a curtain from calculated height
Remove "hidden" points



Meshing mechanism overview

Step 1 - Curtain

Add a “curtain” below the car

- Curtain of points
- Close gap to ground
- Hide wheels

Step 3 - Convert to mesh

- Estimate normals
- Poisson reconstruction
 - Accurate estimation using poisson formula

Step 2 - Fill holes in PC roof

Fill large holes in point cloud roof

- Triggers if Z distribution is low in area
- Projects all points to z plane and fills it

Step 4 - Final Cleaning

- Fix holes in mesh (watertight)
- Smoothen mesh

Challenges

Challenges

Normals

1. Normals were pointing upwards on the sides
2. Normals were pointing inwards in some places
3. Paid library worked
4. Free library worked after more experiments

Smooth but Detailed

1. Too much detail (and noise)
2. Loss in detail
3. Fine Balance

Interior Points

1. Weird “holes” in mesh
2. Cut curtain made them “poles”
3. Figure out camera positions
4. 8 cameras on the outside

Pipeline Architecture

16 steps from raw point cloud to watertight mesh

Pipeline Phases

Cleaning

11-stage noise removal:
voxel downsample,
RANSAC plane removal,
outlier filters, DBSCAN
clustering, raw recovery

Meshing

Screened Poisson
surface reconstruction,
bounding box crop, hole
filling, and optional
smoothing/decimation

Geometry

Feature detection
(mirrors, roof, bumpers,
wheel arches) and
surface generation
(walls, floor, patches)

Repair & Export

Mirror vertex snapping,
watertight check (skips
repair if already
closed), PyMeshFix
repair only when
needed, timestamped
.stl export

Cleaning Pipeline — 11 Stages

Downsample & RANSAC

Voxel grid averaging reduces density. Iterative RANSAC detects and removes floor, ceiling, and wall planes.

Raw Recovery

KDTree-based recovery restores original scan density near confirmed car surfaces. Re-densifies without reintroducing noise.

Outlier Removal

Statistical + radius outlier filters remove isolated points. DBSCAN clustering keeps only the largest connected component.

Floor Band Cleanup

Post-recovery normal floor filter catches remaining ground artifacts.

Percentile-Based Feature Zones

- Dynamic boundaries computed from coordinate percentiles: mirror Y zones, front/rear X boundaries, roof Z height, spike trim threshold

Adaptive Thinning

- Region-specific voxel sizes: mirrors ~0.010m (fine), bumpers ~0.020m, body ~0.040m (coarse)

Wheel Arch & Mirror Detection

- Wheel arches found by scanning Z-layers for X-axis coverage gaps; mirrors identified by Y-axis protrusion analysis with body wall interpolation

Geometry Generation

Roof Patch

Detects sparse roof coverage via grid occupancy. Generates a flat lid at `roof_fill_z` using convex hull + point-in-polygon grid.

Side & End Walls

X-Z grids for sides, Y-Z grids for front/rear. Mirror gaps interpolated from adjacent body wall positions. Gaussian smoothed.

Bumper Patches

Y-Z depth map filled via distance transform and Gaussian-smoothed. Creates curved surfaces that follow the original scan contour.

Floor Cap

Concave hull of XY projection, inset with Shapely buffer. Flat bottom surface at `Z_FLOOR` with downward normals closes the mesh.

YAML-based configuration (pipeline_config.yaml) loaded via PyYAML. Every pipeline stage receives the config dictionary.

Key Parameter Groups

- **Input/Output:** PCD_DIR, OUT_DIR_MESHERS, PICK mode, VERSION tag
- **Cleaning:** voxel_size, RANSAC thresholds, stat/radius outlier params, DBSCAN settings
- **Adaptive Thinning:** per-region voxel sizes (body 0.040, feature 0.010, bumper 0.020)
- **Meshing:** POISSON_DEPTH (6), POISSON_SCALE (1.3), density quantile trimming
- **Intermediate Saving:** per-stage toggle for debugging (.pcd / .stl outputs)

Future Improvements

Thank you!

Check out our website!

